ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab6: Thread Synchronization 1

Submission Deadline: February 25th, 2026 23:59

You need at least 50 points to pass this lab assignment.

1 Basic Concepts

In general, there are two ways for different threads (or proccesses) to communicate with each other: **shared memory** and **message passing**.

Shared-memory:

- Threads running on the same computer or system.
- Apply in shared-memory systems.
- Communication done by using shared-resource such as shared variables, files, database connection.

Message passing:

- Proccesses running on the different computers in a network.
- Apply in distributed-memory systems.
- Communication done by message passing: one proccess sends a message while another proccess receives the message. The sending and receiving operations follow a predefined communication protocol.

In a multithread program, the final output of a program depends on the order of task executions. Without a proper synchronization machinism, the output can be nondeterministic and unexpected. The sequence and timing of task execution are determined by the operating system's scheduler. While the output may be correct at times, it is often incorrect.

Example: Assuming we have a program with two threads (Thread 1 and Thread 2) running on separate cores (core 1 and core 2) simultaneously. Both threads can access and modify a shared variable `count`, initialized as 0. Thread 1 increments `count` by 1, while Thread 2 decrements `count` by 1. If Thread 1 executes first, count will become 1. Subsequently, Thread 2 executes the instruction to decrease `count` by 1. Since `count` is shared, it will be decremented from 1 to 0. On the other hand, if Thread 2 executes first, its value is -1. After Thread 1 executes the instruction to decrease the variable `count`, count should return to 0. Therefore, the expected final output of `count` is 0. However, the actual output depends on the interleaving of instructions executed by the two threads and is subject to the scheduling mechanism of the operating system.

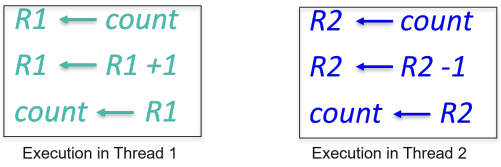

Figure 1: Execution in Thread 1 and Thread 2.

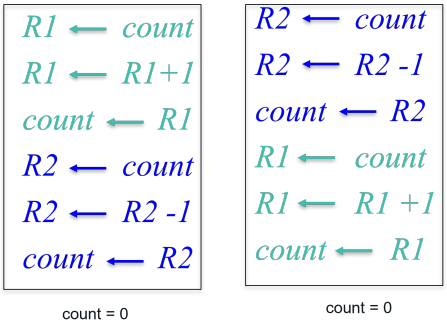

Figure 2: Execution order resulting the final output `count` equal 0.

When a CPU core executes the instruction `count ++` in Thread 1, the executions is divided into three stages, as shown in Figure 1. First, the value of `count` from the memory is loaded into the register, R1, in core 1. The value in the register R1 is manupulated (increased by 1). After that, the new value of R1 is written back to the memory and now, the `count` is updated. This procedure is the same with Thread 2. Figures 2,3 and 4 illustrate different output of the shared variable `count` with different execution order of Thread 1 and Thread 2. To ensure correct behavior of a multithread programs, a synchronization mechanism is needed to control access to the shared variable, or shared resource.

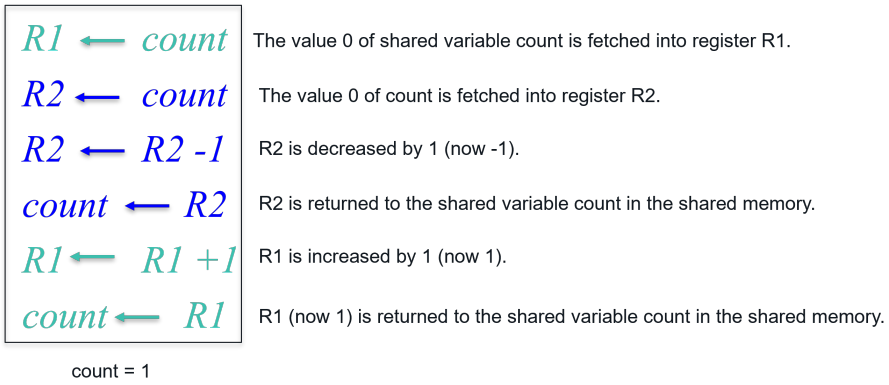

Figure 3: Execution order resulting the final output `count` equal 1.

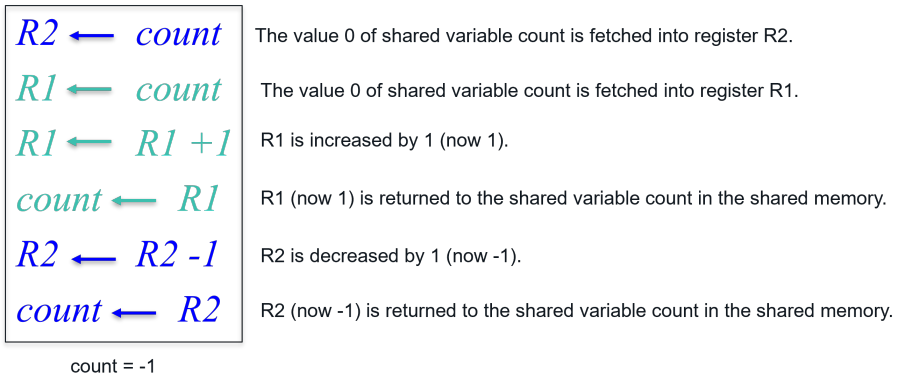

Figure 4: Execution order resulting the final output `count` equal -1.

Project `dataRaceExample` is an implementation of multiple thread execution without a synchronization mechanism. The project creates two threads, both of which can access a shared variable `count`. One thread increases `count` by 10 multiple times, while the other thread decreases `count` by 10 the same number of times. Therefore, the expected output should be 0. However, the actual output varies significantly with each run.

2 Synchronization

Parallel execution is easier when tasks are independent (for example in matrix-matrix multiplication exercise in lab5). However, often these tasks need to cooperate. Cooperation usually means some tasks are writing new values that others must read. To know when a task is finished writing so that it is safe for another to read, the tasks need to synchronize. If they don't synchronize, there is a danger of a data race (explained later), where the results of the program can change depending on how events happen to occur[1].

2.1 Race Condition

A **race condition** occurs when multiple threads access and modify a shared resource (a variable, data structure, same file or database connection) concurrently (e.g., updating the value of a variable at the same time) without proper synchronization. The final value of the share resource becomes unpredictable, depending on which thread "wins" the update race. This can lead to incorrect output or data inconsistency.

A **data race** (the battle for memory control) is a specific type of race condition. Two memory accesses form a data race if they are from different threads to same location, at least one is a write, without the protection of synchronization mechanisms. This is akin to a battle among threads to gain control over the shared memory region. Data race can have even more severe consequences than general race conditions. Data can be overwritten, altered unexpectedly, or even corrupted entirely.

2.2 Critical Section

The solution for **race condition** lies in the concept of **critical section**. A critical section is a **block of code** that accesses a shared resource and cannot be executed by more than one thread at any given time. To avoid race condition problem, the access to the shared-memory area has to be in a critical section protected by a **synchronization mechanism**.

2.3 Mutual Exclusion

Mutual exclusion (or mutex) is perhaps the most prevalent form of coordination in a shared-memory system. Mutual exclusion can be used to restrict access to a critical section to a single thread at a time. It guarantees that one thread "excludes" all other threads while it executes the critical section. Therefore, the mutual exclusion ensures mutually exclusive access to the critical section.

2.4 Synchronization Mechanism

To avoid race condition and to implement critical sections, Java (and almost all programming languages) offers synchronization mechanisms. When a thread wants access to a critical section, it uses one of these synchronization mechanisms to find out if there is any other thread executing the critical section. If not, the thread enters the critical section. Otherwise, the thread is suspended by the synchronization mechanism until the thread that is currently executing the critical section ends it. When more than one thread is waiting for a thread to finish the execution of a critical section, JVM chooses one of them and the rest wait for their turns. Synchronization mechanisms in Java include the following (but not limitted to):

- `Monitor`
- `Lock`
- `Conditional Variable`
- `Semaphores`
- `Barrier`
- `Exchanger`
- `Phaser`

While a synchronization mechanism helps to avoid errors, it introduces overhead to our programs. As a result, the number of concurrent tasks has to be carefully considered. However:

- Big tasks (few number of concurrent tasks) with low intercommunication: reduce overhead but perhaps may not benefit from multiple cores.
- Small tasks (more number of concurrent tasks) with high intercommunication: high overhead but can degrade the performance of the program.

Note that if we want to eliminate overhead and communication among threads, we may want to go back with serial programs.

3 Monitor

The most basic synchronization mechanism in Java is **Monitor**. The Monitor allows threads to have mutual exclusion over a shared resource i.e, only one thread can enter the critical section at a given time. Other threads trying to enter the critical section will be blocked until the first thread releases it.

`Monitor` is implemented by using synchronized keyword. The `synchronized` keyword is used to control concurrent access to a `method` or several `methods`. In addition, `Monitor` allows to define a critical section in a `block of code` in an entire method. All the 'synchronized' used on methods or blocks of code use `an object reference`. Only one thread can execute a method or block of code protected by the same object reference.

3.1 Synchronize a method

Syntax: Add the synchronized keyword in the method declaration to make the method synchronized. For example:

```
synchronized void printNumber(int n){
....
}
```

```
public synchronized void synchronizeSum() {
 sum ++;
}
```

The program below creates two threads without synchronization. When the program is run, two threads execute in a time-interleaved fashion and random order. The output of the program varies each time the code is run.

```
class Number{
   void printNumber(int n){
      int temp = 1;
      for(int i=1;i<=10;i++){
         System.out.println(Thread.currentThread().getName() + ": " + n*i);
         temp = n*temp;
         try{
            Thread.sleep(500);
         }catch(Exception e){
            System.out.println(e);
         }
      }
   }
}
class Thread1 extends Thread{
   Number p;
   Thread1(Number p){
      this.p=p;
   }
   public void run(){
      p.printNumber(2);
   }
}
class Thread2 extends Thread{
   Number p;
   Thread2(Number p){
      this.p=p;
   }
   public void run(){
      p.printNumber(10);
   }
}
public class MonitorSynchronizeMethod{
   public static void main(String args[]){
      Number obj = new Number();//only one object
      Thread1 t1=new Thread1(obj);
      Thread2 t2=new Thread2(obj);
```

```
      t1.start();
      t2.start();
      try {
         t1.join();
      } catch (InterruptedException e1) {
      // TODO Auto-generated catch block
         e1.printStackTrace();
      }
      try {
         t2.join();
      } catch (InterruptedException e) {
      // TODO Auto-generated catch block
         e.printStackTrace();
      }
   }
}
```

If you add the synchronized keyword with the `printNumber()` method to synchronize two threads, they are executed in a way that Thread2 is executed after Thread 1.

3.2 Synchronize several methods

When you use the synchronized keyword with a method, the object reference is implicit. When you use the synchronized keyword in one or more methods of an object, only one execution thread will have access to all these methods. If another thread tries to access any method declared with the synchronized keyword of the same object, it will be suspended until the first thread finishes the execution of the method. In other words, every method declared with the synchronized keyword is a critical section, and Java only allows the execution of one of the critical sections of an object at a time. In this case, the object reference used is the own object, represented by the this keyword. Static methods have a different behavior. Only one execution thread will have access to one of the static methods declared with the synchronized keyword, but a different thread can access other non-static methods of an object of that class. You have to be very careful with this point because two threads can access two different synchronized methods if one is static and the other is not. If both methods change the same data, you can have data inconsistency errors. In this case, the object reference used is the class object. We can use the synchronized keywords with more than one method. In the following snippet, the synchronized keyword is used with two methods `set()` and `get()`.

```
public class EventStorage {

   public synchronized void set(){
   ...
   }
   public synchronized void get(){
   ...
   }
}
```

In this example, `set()` and `get()` are considered as a critical section. Only one thread can call one method at any given time. If one thread is executing `set()` and another thread is trying to call `get()`, the second thread will be suspended. This would reduce the performance of the overall system.

3.3 Synchronize a block of code

Suppose you do not want to synchronize the entire method but a few lines of code in the method, you can also do it by using the `synchronized` keyword. This will take the object as a parameter and work the same as synchronized method. When you use the `synchronized` keyword to protect a block of code, you must pass an object reference as a parameter. Normally, you will use the `this` keyword to reference the object that executes the method. The `exampleMonitor` project contains two examples, one uses the synchronized keyword with entire `pay()` and the other uses `this` keyword to synchronize a block of code in `pay()` method.

```
public void pay(String user, int amount) {
System.out.println(user + " wants to pay " + amount);
     int fee = 5;
     int total = amount + fee;

     int newBalance;
     synchronized (this) {
         if (balance >= total) {
             balance -= total;       // shared state update
             newBalance = balance;   // capture correct value
       } else {
           System.out.println("Not enough money for " + user);
           return;
       }
      }
      // More work outside the synchronized block
           System.out.println(user + " paid " + amount +
                  " (fee " + fee + "), new balance = " + newBalance);
}
```

If you have two independent attributes in a class shared by multiple threads, you can synchronize access to each attribute. It would not be a problem if one thread accesses one of the attributes and another thread accesses a different attribute simultaneously. The following snippet of code is another example of using `synchronized` to protect a block of code. The block then becomes a critical section which is assessed by no more than one thread at a time.

```
public class SafeParkingStats {

/**
* This two variables store the number of cars and motorcycles in the parking
*/
   private long numberCars;
   private long numberMotorcycles;

   /**
   * Two objects for the synchronization. ControlCars synchronizes the
   * access to the numberCars attribute and controlMotorcycles synchronizes
   * the access to the numberMotorcycles attribute.
   */
   private final Object controlCars, controlMotorcycles;

   public void carComeIn() {
      synchronized (controlCars) {
        numberCars++;
```

```
    }
  }

  public void motoComeIn() {
    synchronized (controlMotorcycles) {
      numberMotorcycles++;
    }
  }
}
```

In this example, there are two objects i.e, `controlCar` and `controlMotorcycles` and two attributes i.e, `numberCars` and `numberMotorcycles`. The `numberCars` attribute is protected by the `controlCar` object and the `numberMotorcycles` attribute is protected by the `controlMotorcycles` object. In the `carComeIn()` method, `numberCars` is increased and in `motoComeIn()` method, `controlMotorcycles` is increased. If we synchronize by using the synchronized keyword with a method, when one thread is executing `carComeIn()`, no thread can increase the `numberMotorcycles` attribute. The two objects in this example are considered as two different critical sections. Therefore, these two blocks can be executed simultaneously by different threads. While one thread is increasing `numberCars`, another thread can increase `numberMotorcycles`. However, no two threads will be able to modify the same attribute at the same time.

3.4 Using condition in synchronized code

In Java, `wait()`, `notify()`, and `notifyAll()` can be used inside `synchronized` code i.e, a synchronized method or synchronized block of code. When a thread call a `wait()` method, JVM puts the thread to sleep and the thread releases the lock. To wake up the thread, we have to call `notify()` or `notifyAll()` inside the synchronized block. Then, the lock is reacquired. Note that a thread cannot call `wait()` outisde a synchronized block of code. If this happens, JVM will throw an `IllegalMonitorStateException` exception.

Let see one example in the `producerConsumer` package, refer to Figure 6. In this example, the producer and the consumer share a queue buffer. The queue has a size of 10, meaning that there will not be more than 10 items in the queue at any moment. The producer is supposed to produce 100 items and put the in the queue buffer. The consumer is supposed to consume the items in the queue and it can consume upto 100 items. However, the queue size is only 10. This means the producer cannot put more than 10 items at once and the consumer cannot remove more than 10 items at once, neither. Therefore, the queue buffer has to be controlled in such a way that the producer cannot put more item in the queue when the queue is full and the consumer cannot consume any item when the queue is empty. We can use `wait()`, `notify()` methods for this purpose. The following method will be called by the producer to put one item in the queue.

```
public synchronized void put(){
    while (storage.size()==maxSize){ // When the queue is full, the producer stops putting an
        item into the queue
      try {
           wait(); // Stop by sleeping
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
    }
    storage.add(new Date());
    System.out.printf("Put: %d\n",storage.size());
```

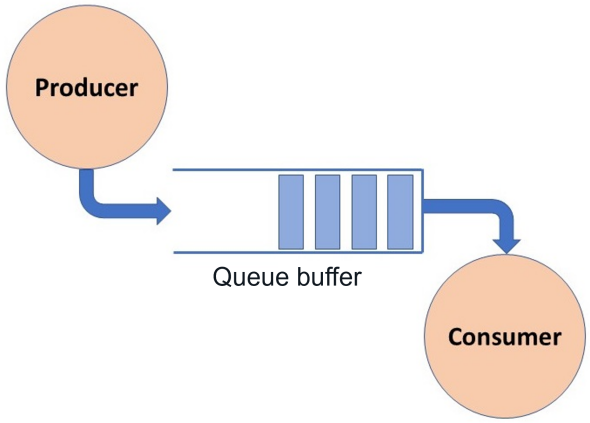

Figure 5: Producer/Consumer problem.

```
      notify(); // Wakes up when there is a room in the queue
}
```

When the queue is full, the producer (thread) will release the lock and forced to sleep. When there is a room in the queue, the thread is waken up by `notify()` method and a new item is put in the queue. The similar procedure is implemented with the consumer.

```
public synchronized void remove(){
   while (storage.size()==0){// When the queue is full, the producer stops removing an item from
       the queue
      try {
         wait(); // Stop by sleeping
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }
   String item=storage.poll().toString();
   System.out.printf("Remove: %d\n",storage.size());
   notify();// Wakes up when there is an item in the queue
}
```

When the queue is empty, the consumer is forced to sleep by `wait()` method. When there is an item available in the queue, it is waken up by `notify()` inside the synchronized code block. The consumer removes the first item in the queue by `poll` method. In this project, the producer produces 100 items, therefore, it has to produce 10 times; each time it produces 10 items. The similar procedure applies to the consumer.

4 Lock

Lock is another basic synchronization machanism for ensuring mutual exclusion. It is more flexible than Monitor which uses the synchronized keyword because Lock allows the programmers to lock and unlock the critical section from anywhere in the code.

4.1 Reentrant Lock

In `Lock` synchronization mechanism, only one thread can hold a lock at a time. A thread acquires (locks) a lock when it executes a `lock()` method and releases (unlocks) the lock when it executes an `unlock()` method. A thread is well-formed if:

- Each critical section is associated with a unique `Lock` object.
- The thread calls that object's `lock()` method when it is trying to enter the critical section.
- The thread calls the `unlock()` method when it leaves the critical section.

Lock is implemented by the `Lock` interface and its implementation classes. The `Lock` interface provides two important methods: `lock()` and `unlock()`. The basic class which implements Lock is `ReentrantLock` class. In Java, the implementation of `Lock` should follow the following structured way:

```
.....
private static ReentrantLock lock = new ReentrantLock();

lock.lock();
try {
   // BLOCK OF CODE IN CRITICAL SECTION
   } finally {
      lock.unlock();
}
```

This is to ensure that the lock is acquired before entering the **try** block and that the lock is released when the control leaves the block even if some statements in the block throws an unexpected exception. If `unlock()` is not called, an Exception is thrown, the lock will not be released and you will have a **deadlock**. One example is in the `reentrantLock` project. The task is implemented with Lock synchronization mechanism:

```
public class Task implements Runnable {

   private static ReentrantLock lock = new ReentrantLock();
   private String name;

   public Task(String name) {
      this.name = name;
   }

   public void run() {
      lock.lock();
      try {
         System.out.println( name + ": Running the task");
         Work.doTask();
         System.out.println("Task: " + name + ": The execution has finished");
      } finally {
         lock.unlock();
      }
   }
}
```

In the main, we create 5 Tasks and use executor framework to run the 5 Tasks. Without synchronization, the output of the program is different from run to run and Threads are executed without any order and are overlapping their operations. One possible output is the following:

```
Task 0: Running the task
Task 1: Running the task
Task 2: Running the task
Task 3: Running the task
Task 4: Running the task
pool-1-thread-1: Working 8 seconds
pool-1-thread-3: Working 5 seconds
pool-1-thread-4: Working 7 seconds
pool-1-thread-5: Working 3 seconds
pool-1-thread-2: Working 5 seconds
pool-1-thread-5: Finished
Task 4: The execution has finished
pool-1-thread-3: Finished
Task: Task 2: The execution has finished
pool-1-thread-2: Finished
Task 1: The execution has finished
pool-1-thread-4: Finished
Task 3: The execution has finished
pool-1-thread-1: Finished
Task 0: The execution has finished
```

When threads are synchronized, one task is executed when another thread finishes its execution:

```
Task 0: Running the task
pool-1-thread-1: Working 5 seconds
pool-1-thread-1: Finished
Task 0: The execution has finished
Task 1: Running the task
pool-1-thread-2: Working 2 seconds
pool-1-thread-2: Finished
Task 1: The execution has finished
Task 2: Running the task
pool-1-thread-3: Working 9 seconds
pool-1-thread-3: Finished
Task 2: The execution has finished
Task 3: Running the task
pool-1-thread-4: Working 2 seconds
pool-1-thread-4: Finished
Task 3: The execution has finished
Task 4: Running the task
pool-1-thread-5: Working 3 seconds
pool-1-thread-5: Finished
Task 4: The execution has finished
```

4.2 ReadWrite Lock

The most improvement of `Lock` is ReadWrite Lock. It is implemented by using `ReadWriteLock` interface and `ReentrantReadWriteLock` class. This class has two locks: read operation and write operation. Multiple threads can perform read operation simultaneously. However, only one thread can perform write

operation at any given time. While a thread is doing write operation, all other threads cannot perform neither read nor write operations. Since `ReadWriteLock` allows access by multiple reader threads, it enhance the concurrency of the program.

The `readWriteLock` package demonstrates the use of ReadWriteLock to simulate the readers and writers that have access to the same shared PriceInfo data. The readers read the prices while the writer try to modify the prices. The readers and writers sychronize data access with ReadWrite Lock. The core implementations of two locks are:

```
public double getPrice1() {
   lock.readLock().lock();
   double value=price1;
   lock.readLock().unlock();
   return value;
}

public double getPrice2() {
   lock.readLock().lock();
   double value=price2;
   lock.readLock().unlock();
   return value;
}

public void setPrices(double price1, double price2) {
   lock.writeLock().lock();
   System.out.printf("%s: PricesInfo: Write Lock Acquired.\n", new Date());
   try {
      TimeUnit.SECONDS.sleep(10);
   } catch (InterruptedException e) {
      e.printStackTrace();
   }
   this.price1=price1;
   this.price2=price2;
   System.out.printf("%s: PricesInfo: Write Lock Released.\n", new Date());
   lock.writeLock().unlock();
}
```

Part of the output of one execution is:

```
Tue Feb 21 22:48:02 CET 2023: PricesInfo: Write Lock Acquired.
Tue Feb 21 22:48:12 CET 2023: PricesInfo: Write Lock Released.
Tue Feb 21 22:48:12 CET 2023: Writer: Prices have been modified.
Tue Feb 21 22:48:02 CET 2023: Thread-4: Reader: Price 2: 7.350256
Tue Feb 21 22:48:12 CET 2023: Thread-4: Reader: Price 1: 1.749456
Tue Feb 21 22:48:12 CET 2023: Thread-4: Reader: Price 2: 7.350256
.....
Tue Feb 21 22:48:12 CET 2023: Thread-2: Reader: Price 1: 1.749456
Tue Feb 21 22:48:12 CET 2023: Thread-2: Reader: Price 2: 7.350256
Tue Feb 21 22:48:12 CET 2023: Thread-2: Reader: Price 1: 1.749456
Tue Feb 21 22:48:12 CET 2023: Thread-2: Reader: Price 2: 7.350256
Tue Feb 21 22:48:12 CET 2023: Writer: Attempt to modify the prices.
Tue Feb 21 22:48:12 CET 2023: PricesInfo: Write Lock Acquired.
Tue Feb 21 22:48:22 CET 2023: PricesInfo: Write Lock Released.
Tue Feb 21 22:48:22 CET 2023: Writer: Prices have been modified.
```

```
Tue Feb 21 22:48:22 CET 2023: Writer: Attempt to modify the prices.
Tue Feb 21 22:48:22 CET 2023: PricesInfo: Write Lock Acquired.
Tue Feb 21 22:48:32 CET 2023: PricesInfo: Write Lock Released.
Tue Feb 21 22:48:32 CET 2023: Writer: Prices have been modified.
```

It can be seen from the output that while the writer has acquired the write lock, none of the readers can read the data. Once the writer has released the lock, the readers again can access to the price information and print the new modified prices.

5 Exercises

5.1 Exercise 1 (25 pts)

- Run the code in section 3.1 several times and include the outputs in your report.
- Do you get the same output for all runs?
- Add synchronized keyword to synchronize two threads. Run the new code several times and include outputs in your report. Do you get the same output for all runs?

5.2 Exercise 2 (25 pts)

Refer to project `lab6` for this exercise.

- Run the package `producerConsumer` and include the output in your report.
- Which synchronization mechanism(s) and method(s) are used to synchronize the Producer thread and the Consumer thread in this package?
- Remove the synchronization of Producer and Consumer threads and explain how a data race occurs.

5.3 Exercise 3 (25 pts)

The package `raceCondition` implements 10 increasing tasks and 10 decreasing tasks. Each increasing task increases a shared varaiable `count` by 1, while each decreasing tasks decreases the variable `count` by 1. The expected output of `count` is 0. However, the actual output of `count` varies with each run and is never 0.

- Explain why a data race occurs.
- Where are the critical sections?
- Fix the data inconsistency problem using `Monitor`.
- Fix the data inconsistency problem using `Lock`.

5.4 Exercise 4 (25 pts)

The `parking` project simulates a parking system. The system consists of multiple sensors to track the number of cars and the number of motorcycles coming in and out in a parking area. Each sensor is managed by one thread.

There are two implementations of this parking system: a safe version (in `safe` package) and an unsafe version (in `unsafe` package). The classes in the `unsafe` package do not have any synchronization mechanism

and therefore have data race problem. On the other hand, the classes in `safe` package use **Monitor** to solve the data race problem. In both implementations, variables `numberCars` and `numberMotorcycles` are used to keep track of the number of cars and motorcycles. The `ParkingCash` contains a shared variable `cash` that will be increased by one every time one vehicle going out.

- Run the two implementations: `safe` and `unsafe` and include the output in your report
- What would be the expected output of `numberCars` and `numberMotorcycles`?
- Discuss the data race issue focusing on the use of the `sychronized` keyword.

6 What To Submit

Complete the the exercises in this assignment. Then, put your codes along with lab6_report file (Markdown or pdf file) inside the lab6 directory of your repository. Commit your changes and remember to check the GitHub website to make sure all files have been submitted.

7 References

1. David A Patterson and John L. Hennessy, Computer organization and design : the hardware/software interface, 5th Edition, 2014.
2. González, Javier Fernández. Java 9 Concurrency Cookbook. Packt Publishing Ltd, 2017.